
PROJET ROVER

Commande PI des vitesses des roues du rover

AERO 2 – 2025/2026



-
- Objectif :**
- Commande de vitesse d'un moteur à courant continu (contrôleur P)
 - Enregistrement des logs des vitesses, et traçage des courbes
 - Commande PI des quatre vitesses
 - Faire des mouvements plus complexes
-

1. Création d'un répertoire et configuration de l'environnement

Avant de commencer le codage sous VSCode, assurez-vous de la bonne configuration de l'**environnement de travail**.

- Dans le répertoire « Rover », créer un sous-répertoire de travail « Commande PI ».
- Dans VSCode, cliquer sur *Explorer* et ouvrir le répertoire (*folder*) « Commande PI ».
- Cliquer sur la barre de recherche en haut de l'écran puis sur « *Show and Run Commands* ».
- Taper « *MicroPico : Initialize MicroPico Project* » pour configurer l'environnement de travail avec MicroPython. Des fichiers « .micropico » et « .vscode » sont ajoutés dans votre répertoire de travail. Ils contiennent les informations de configuration, quelques réglages locaux de VSCode pour ce projet (police, couleurs, taille des lignes), etc.... Ne pas les supprimer.

2. Tester les codes du TP5

Récupérer les codes de TP5, et les tester.

- Copier dans le répertoire votre dernier programme « main.py » et le module « motor_driver_lib ».
 - *Firmware* (fichier en .uf2) : Si ce n'est pas encore fait, charger le *firmware* et copier le dans la carte comme indiqué sur le site *micropython.org*.
 - Cliquer sur « *Toggle Mpy FS* » pour afficher, dans l'Explorer, les fichiers importés dans Pico W. Ces fichiers apparaîtront dans l'onglet « *Mpy Remote Workspace* ».
 - Exécuter le code sur Raspberry Pi Pico W.
- Deux façons sont possibles pour exécuter le script Python dans le microcontrôleur :
1. **Garder le câble USB connecté au PC, allumer la batterie, cliquer sur « main.py » de votre répertoire de travail, et lancer « Run ».** Dans ce cas, le code n'est pas importé sur le microcontrôleur.
 2. Ou bien, pour une exécution de façon autonome, transférer les fichiers « main.py » et « motor_driver_lib » du répertoire de travail dans le microcontrôleur. Pour cela, faire un clic droit sur les fichiers, puis sélectionner « *Upload file to Pico* ». Débrancher le rover de votre ordinateur, puis allumer la batterie.

3. Rappel des fonctions du code fourni de motor_driver_lib.py

def set_motor_parameters(): configure le contrôleur des moteurs installé sous le Rover.

def read_total_encoder_counts(): retourne une liste de 4 éléments, les nombres d'impulsions (*nombres de ticks*) comptées par les 4 encodeurs.

def read_encoder_deltas(): compte le nombre d'impulsions des 4 roues sur un intervalle de temps de 10ms.

def calculate_rpm(ticks, TICKS_PER_REVOLUTION, SAMPLE_INTERVAL): calcule la vitesse rpm d'une roue.
ticks est le nombre d'impulsions sur un intervalle de 10 ms.

def control_motor_pwm(p1, p2, p3, p4): permet de commander par PWM des 4 moteurs. Les quatre valeurs p1 à p4 permettent de modifier le rapport cyclique du signal PWM envoyé à chaque moteur indépendamment. En variant les rapports cycliques, les tensions moyennes varient, donc les vitesses des roues varient.

def control_motor_voltage(v1, v2, v3, v4): commande les 4 moteurs en spécifiant la tension moyenne sur chaque moteur (en volts).

def voltage_to_pwm(voltage): convertit une valeur en volts en une valeurs PWM. Sachant que :
0 V -> PWM=0, et 7.34 V -> PWM=2000.

4. Commande de vitesse d'une seule roue

Ajouter dans le fichier main.py la fonction suivante :

```
# -----  
# Contrôle en vitesse d'une seule roue  
# -----  
# --- Paramètres du PI (à ajuster selon ton identification) ---  
Kp = 0.02      # [V/rpm]  
Ki = 0.02      # [V/(rpm.s)]  
DT = 0.020     # [s] Période d'Echantillonnage (20 ms)  
  
V_MAX = 7.34   # V  
  
def control_vitesse(consigne_V1, duree=5.0):  
    """  
    consigne_V1 : consigne de vitesse en rpm de la roue 1  
    duree : durée du test (s)  
    """  
    print(f"Control active: Consigne = {consigne_V1} rpm, Duree = {duree}s")  
  
    t_start = time.time() # instant de départ  
  
    while (time.time() - t_start) / 1000.0 < duree:  
        # Lire l'incrément d'encodeur sur 20 ms  
        deltas = motor.read_encoder_deltas()  
        V1 = motor.calculate_rpm(deltas[0])  
  
        # Calcul de l'erreur  
        error = consigne_V1 - V1  
  
        # Calcul de la tension de commande (PI)  
        u = Kp * error  
  
        # Saturation à -7.34 à 7.34 V  
        if u > V_MAX:  
            u = V_MAX  
        elif u < -V_MAX:  
            u = -V_MAX
```

```
# Appliquer la tension sur le moteur choisi
voltages = [u , 0.0, 0.0, 0.0]
motor.control_motor_voltage(*voltages)

print(f"Consigne: {consigne_V1:.1f} rpm, Measure: {V0:.2f} rpm, u={u:.2f} V")
time.sleep(DT)
```

Cette fonction utilise la commande P pour contrôler la vitesse de la première roue.

- Si ce n'est pas encore fait, compléter les deux fonctions `voltage_to_pwm()` et `calculate_rpm()` du TP5.
- Tester la fonction `control_vitesse()` dans votre programme principal pour asservir la vitesse à une **vitesse de consigne de 250 rpm**.

5. Enregistrer les logs des vitesses dans un fichier CSV

Pour pouvoir tracer les courbes des vitesses en fonction du temps, il est intéressant d'enregistrer les logs des données des capteurs (vitesses de rotation des roues).

- Ajouter dans le fichier `main.py` (ou créer un autre module `.py`) les deux fonctions suivantes :

```
import os
# =====
# Fonctions utilitaires de LOGGING dans le microcontrôleur Pico W
# =====
LOG_DIR = "logs" # Répertoire pour mettre le de log
LOG_FILE = LOG_DIR + "/speed_log0.csv" # Chemin d'accès vers le fichier log

def init_logging():
    """Créer le dossier et le fichier CSV si besoin."""
    if LOG_DIR not in os.listdir():
        os.mkdir(LOG_DIR)

    with open(LOG_FILE, "w") as f:
        f.write("time_ms, V1_rpm, V2_rpm, V3_rpm, V4_rpm\n")

def append_log(temps, V1, V2, V3, V4):
    """Ajoute une ligne dans le fichier CSV."""
    with open(LOG_FILE, "a") as f:
        f.write(f"{temps}, {V1}, {V2}, {V3}, {V4}\n")
```

- Faire appel à `init_logging()` avant la boucle `while` pour créer le fichier CSV.
Faire appel à `append_log()` dans la boucle pour sauvegarder les vitesses à chaque période d'échantillonnage.
La première colonne `time_ms` est le temps écoulé entre l'instant de mesure et `t_start`. Utiliser la fonction `time.ticks_ms()`.
- Lancer le programme pour faire tourner les roues. Un répertoire nommé « logs » contenant les fichiers de log serait créé dans le microcontrôleur (voir l'onglet « *Mpy Remote Workspace* »).
- Récupérer les données du fichier CSV.
- Tracer la courbe de vitesse (de préférence sous Matlab, sinon sous Excel).

6. Commande PI des quatre roues

- Adapter votre code pour faire un contrôle PI de la première roue, au lieu du contrôle P. Ajouter la composante intégrale. Pour cela, vous pouvez créer une variable initialisée à 0, puis y ajouter l'erreur dans la boucle while à chaque période d'échantillonnage, puis calculer la commande PI.
- Tester votre commande, tracer la courbe de vitesse et essayer d'optimiser votre contrôleur PI avec un choix plus approprié des paramètres K_p et K_i .
- Modifier votre code pour contrôler les vitesses des quatre roues. Tracer les courbes de ces vitesses.
- Pour finir, essayer de réaliser d'autres mouvements plus complexes avec le Rover (trajectoire carrée, circulaire...). Essayer d'améliorer les performances tout en gardant des bons paramètres K_p , K_i pour la commande PI.